

基于 FPGA 的 16 位 RISC_V_CPU 设计

张宇轩 12210924 邓卜凡 12211122

目录

Abstract	1
Introduction	1
Methodology	1
系统架构设计	1
指令集架构设计	2
Results and Analysis	3
Conclusion	7
Appendix	8
VHDL 模块实现与代码	8

Abstract

本项目设计了一款基于 FPGA 的 16 位 RISC-V CPU。该设计采用 VHDL 语言在 Xilinx Vivado 平台上实现，通过模块化方法构建，涵盖程序计数器（PC）、随机访问存储器（RAM）、指令解码器、控制单元、寄存器文件和算术逻辑单元（ALU）等关键组件。报告将详细阐述系统架构、指令集架构、仿真结果以及上板验证结果。

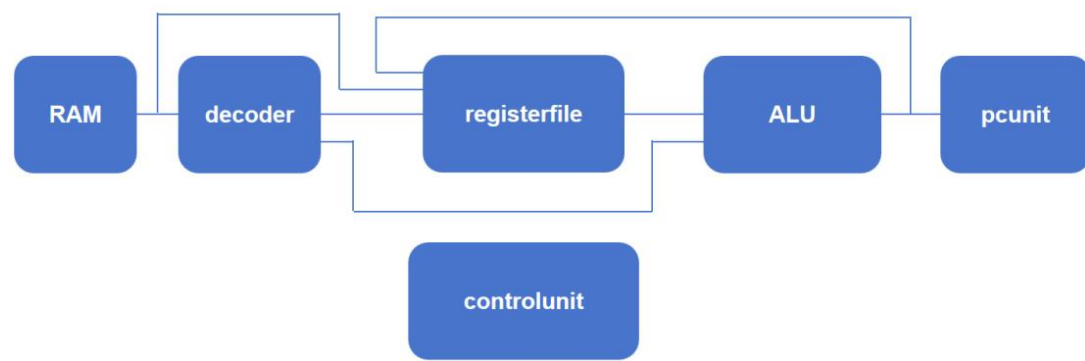
Introduction

随着嵌入式系统和物联网技术的飞速发展，对高效、灵活且可定制的处理器的需求日益增长。RISC-V 架构以其开源、模块化和可扩展的特性，为学术研究和商业应用提供了理想的处理器设计平台。基于现场可编程门阵列（FPGA）的硬件实现，能够快速验证和迭代 CPU 设计，降低开发成本和风险，本文旨在探索利用 FPGA 和 VHDL 设计一款简化的 16 位 RISC-V CPU，模拟其执行汇编指令的能力，特别是针对 1 到 N 的累加和累乘运算，为更复杂的处理器开发积累经验。

Methodology

系统架构设计

设计采用模块化方法，CPU 架构如下图所示。在上板验证时，我们使用数码管来显示二进制的计算结果。



PC Unit：负责提供下一条指令地址并处理分支跳转。初始状态指向程序起始地址，通常在执行完一条指令后递增以指向后续指令，遇到跳转指令时更新为指定目标地址。

RAM：存储指令和数据。设计中配置为 256 个 16 位地址空间，支持读写操作，满足程序存储和数据交互需求。

Decoder：对指令进行解码，提取操作码和操作数等关键信息，生成控制信号以指导后续执行流程。

Control Unit：协调各阶段执行顺序。作为控制中心，根据解码结果产生时序控制信号，确保数据正确流动和指令按序执行，涵盖指令取指、解码、执行、写回等阶段控制。

Register File：包含 8 个 16 位寄存器，用于暂存操作数和计算结果，快速响应 CPU 数据需求，减少数据访问延迟。

ALU：执行算术和逻辑运算，如加法、减法、与、或、异或等操作，支持数据处理核心功能。

指令集架构设计

Instruction	Form	Implementation	Writes Register	OPCODE
ADD	RRR	$rD = rA + rB$	c: 1/0 = signed/unsigned	0000
SUB	RRR	$rD = rA - rB$	c: 1/0 = signed/unsigned	0001
NOT	RRU	$rD = \text{not } rA$	c: N/A	0010
AND	RRR	$rD = rA \text{ and } rB$	c: N/A	0011
OR	RRR	$rD = rA \text{ or } rB$	c: N/A	0100
XOR	RRR	$rD = rA \text{ xor } rB$	c: N/A	0101
LSL	RRR	$rD = rA \ll rB$	c: N/A	0110
LSR	RRR	$rD = rA \gg rB$	c: N/A	0111
CMP	RRR	$rD = \text{cmp}(rA, rB)$	c: 1/0 = signed/unsigned	1000
B	UI(8)	$PC = rA \text{ or } 8\text{-bit immediate}$	c: 1/0 = rA/8-bit immediate	1001
BEQ	URR	$PC = rA \text{ conditional } rB$	c: N/A	1010
IMMEDIATE	RI(8)	$rD = 8\text{-bit immediate}$	c: 1/0 = upper/lower 8-bits	1011
LOAD	RRU	$rD = \text{memory}(rA)$	c: N/A	1100
STORE	URR	$\text{Memory}(rA) = rB$	c: N/A	1101

Form	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RRR	C	opcode				rD			rA			rB		U		
RRU	C											U				
URR	C															
RI(8)	C	opcode				rD			8-bit immediate							
UI(8)	C															

指令功能和格式：采用固定 16 位指令长度，操作码占 4 位，支持算术运算（加、减、乘）、逻辑运算（与、或、异或、非）、数据传输（读、写内存）、控制流（无条件跳转、条件跳转）等基本操作。指令的格式如上图所示。

指令执行流程：在取指阶段，PC 单元提供当前指令地址，从 RAM 中读取指令；解码阶段，Decoder 模块解析指令，提取操作码和操作数信息，生成控制信号；执行阶段，ALU 根据操作码对来自寄存器文件的操作数进行运算，对于数据传输指令则涉及 RAM 的读写操作；写回阶段，将运算结果写入寄存器文件或更新 PC 值以改变程序流程。整个过程由控制单元协调，确保各阶段按序进行，实现指令的完整执行。



Results and Analysis

仿真结果：

Test 1

在测试一中，我们测试累加的功能，让 N 为 5，预计得到的结果为 15。

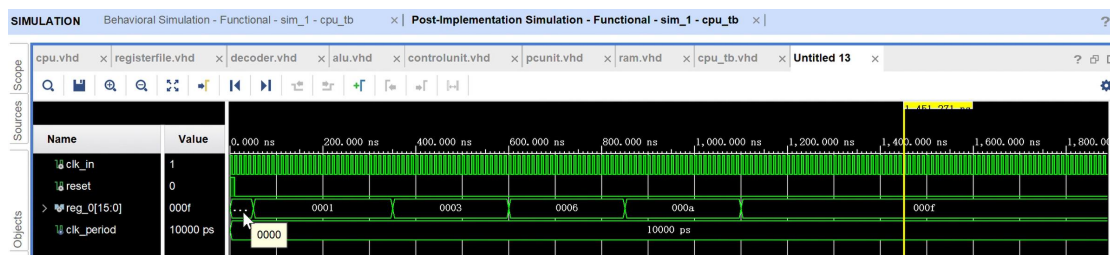
在 RAM 中存入下图中的指令，可以看到 N 的值设置为 5。

```

IMPLEMENTS DESIGN - xc7a100tcs9324-1
Project Summary | Device | cpu.vhd | registerfile.vhd | decoder.vhd | alu.vhd | controlunit.vhd | pcunit.vhd | ram.vhd | cpu_tb.vhd
D:/DSD/cpu/c9u.sr9s/sources_1/imports/sources/ram.vhd
47 -- ; 循环结束后代码
48
49 architecture Behavioral of ram is
50     type ram_array is array (0 to 255) of STD_LOGIC_VECTOR (15 downto 0);
51     signal ram : ram_array := ( -- RAM preload instructions
52         -----累加-----
53         '0' & "1011" & "000" & x"01", -- IMMEDIATE rA, 1
54         '0' & "1011" & "001" & x"02", -- IMMEDIATE rB, 2
55         '0' & "1011" & "010" & x"05", -- IMMEDIATE rC, N
56         '0' & "1011" & "100" & x"01", -- IMMEDIATE r4, 1
57         '0' & "1011" & "101" & x"0B", -- IMMEDIATE r5, End_Label
58         '0' & "1011" & "110" & x"06", -- IMMEDIATE r6, Loop_start_Label
59         '0' & "0000" & "000" & "000" & "001" & "00", -- ADD rA, rA, rB
60         '0' & "1000" & "011" & "001" & "010" & "00", -- CMP rB, rC
61         '0' & "1010" & "000" & "101" & "000" & "00", -- BEQ rD, 1, End_Label
62         '0' & "0000" & "001" & "001" & "100" & "00", -- ADD rB, rB, r4(1)
63         '1' & "1001" & "000" & "110" & "000" & "00", -- B Loop_Start_Label
64

```

进行 Post-implementation simulation - Functional，得到下图的结果。可以看到 reg_0（我们设计的存放累加结果的寄存器）最终的为 000f，转换为 10 进制即为 15，与期望得到的结果相同。



Test 2

在测试二中，我们测试累乘的功能，让 N 为 6，预计得到的结果为 720。

在 RAM 中存入下图中的指令，可以看到 N 的值设置为 6。

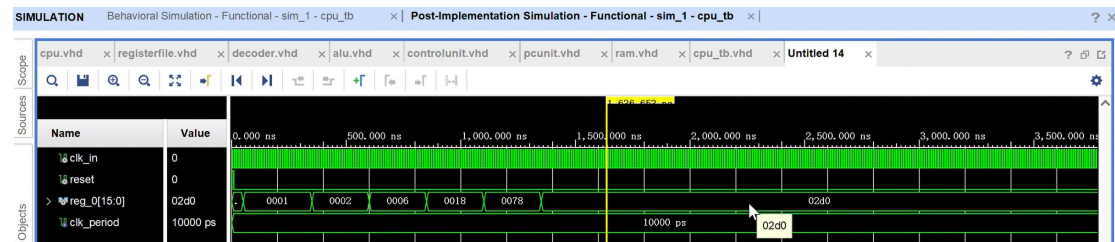
```

cpu.vhd | registerfile.vhd | decoder.vhd | alu.vhd | controlunit.vhd | pcunit.vhd | ram.vhd | cpu_tb.vhd | Untitled 14
D:/DSD/cpu/c9u.sr9s/sources_1/imports/sources/ram.vhd
59 -- '0' & "0000" & "000" & "000" & "001" & "00", -- ADD rA, rA, rB
60 -- '0' & "1000" & "011" & "001" & "010" & "00", -- CMP rB, rC
61 -- '0' & "1010" & "000" & "101" & "000" & "00", -- BEQ rD, 1, End_Label
62 -- '0' & "0000" & "001" & "001" & "100" & "00", -- ADD rB, rB, r4(1)
63 -- '1' & "1001" & "000" & "110" & "000" & "00", -- B Loop_Start_Label
64
65 -----累乘-----
66 '0' & "1011" & "000" & x"01", -- IMMEDIATE rA, 1
67 '0' & "1011" & "001" & x"02", -- IMMEDIATE rB, 2
68 '0' & "1011" & "010" & x"06", -- IMMEDIATE rC, N
69 '0' & "1011" & "100" & x"01", -- IMMEDIATE r4, 1
70 '0' & "1011" & "101" & x"0B", -- IMMEDIATE r5, End_Label
71 '0' & "1011" & "110" & x"06", -- IMMEDIATE r6, Loop_start_Label
72 '0' & "1110" & "000" & "000" & "001" & "00", -- MUL rA, rA, rB
73 '0' & "1000" & "011" & "001" & "010" & "00", -- CMP rB, rC
74 '0' & "1010" & "000" & "101" & "000" & "00", -- BEQ rD, 1, End_Label
75 '0' & "0000" & "001" & "001" & "100" & "00", -- ADD rB, rB, r4(1)
76 '1' & "1001" & "000" & "110" & "000" & "00", -- B Loop_Start_Label

```

进行 Post-implementation simulation - Functional，得到下图的结果。可以

看到 reg_0（我们设计的存放累加结果的寄存器）最终的为 02d0，转换为 10 进制即为 720，与期望得到的结果相同。



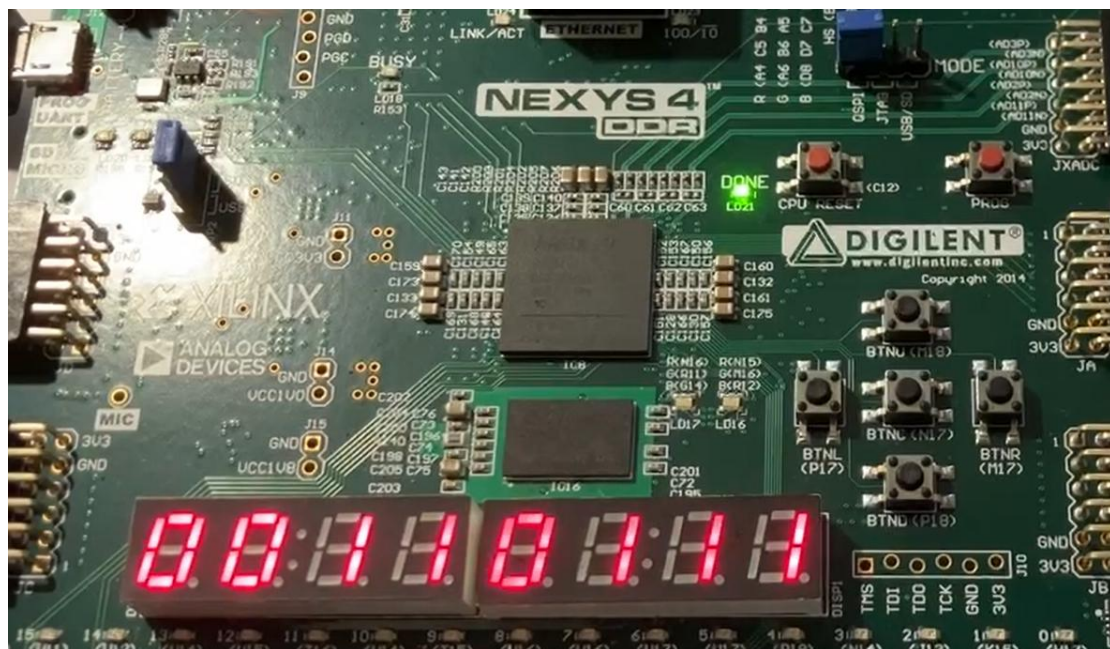
实际结果：

在上板验证时，我们使用数码管来显示二进制的计算结果。完整的测试过程可参考提交的演示视频。

Test 1

我们测试累加的功能，让 N 为 10，预计得到的结果为 55，其对应的二进制数字为 110111。

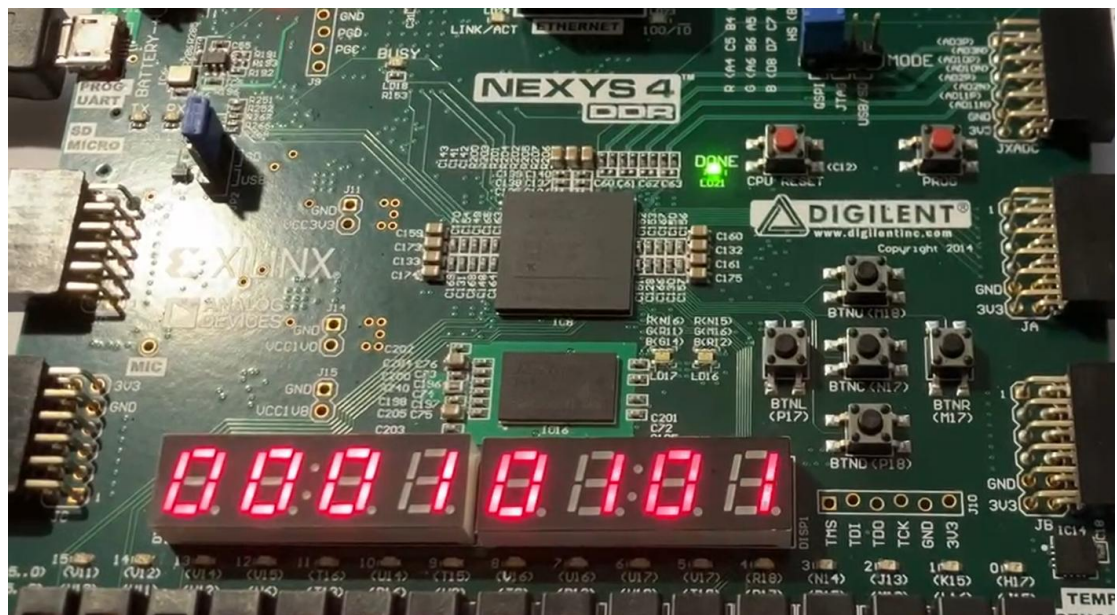
下图为测试的结果。



Test 2

我们测试累加的功能，让 N 为 6，预计得到的结果为 21，其对应的二进制数字为 10101。

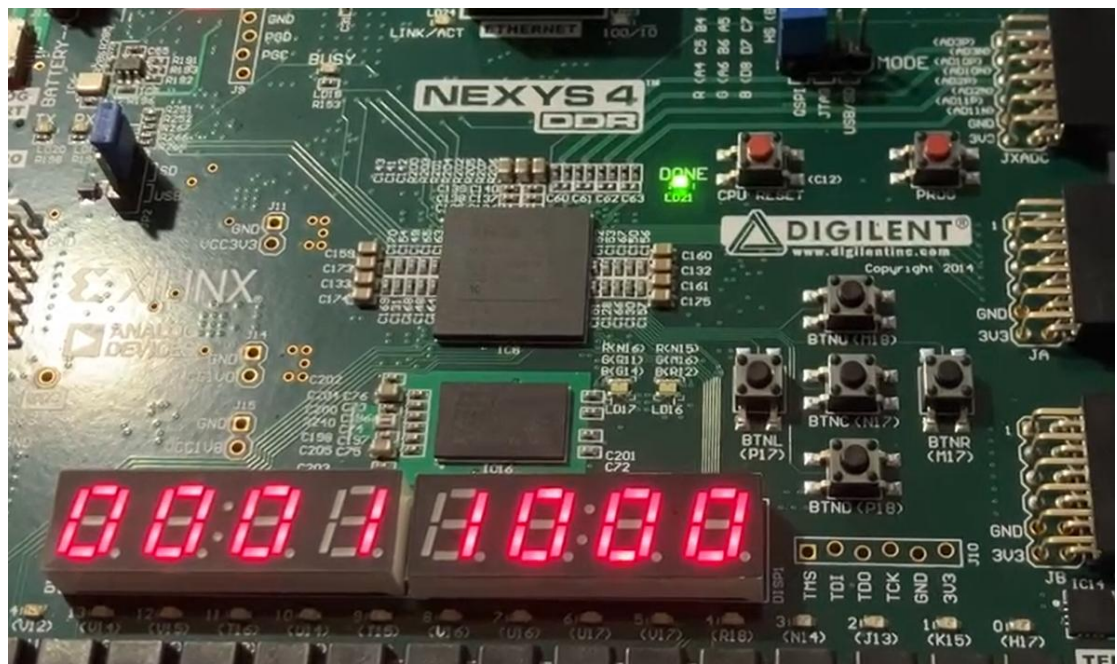
下图为测试的结果。



Test 3

我们测试累乘的功能，让 N 为 4，预计得到的结果为 24，其对应的二进制数字为 11000。

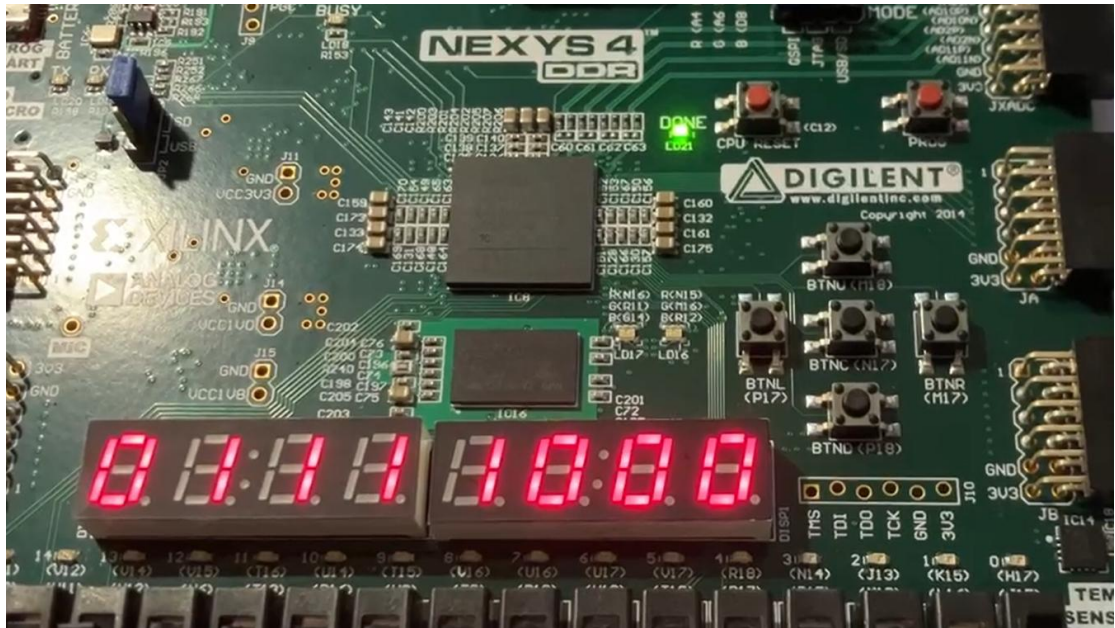
下图为测试的结果。



Test 4

我们测试累乘的功能，让 N 为 5，预计得到的结果为 120，其对应的二进制数字为 1111000。

下图为测试的结果。



结果分析：

对于上面的结果，我们可以看到不管是仿真还是上板验证，cpu 计算得到的值都与真实值相符合，说明 cpu 可以通过改变 RAM 中的指令计算不同 N 对应的累加/累乘结果。

性能评估：

我们设计的寄存器位宽为 16 位，位数限制了计算结果的范围。16 位无符号二进制数转换为十进制的范围为 0~65535，说明对于累加运算，cpu 最多可以进行从 1 到 361 的累加（结果为 65341）；对于累乘运算，cpu 最多可以进行从 1 到 8 的累乘（结果为 40320），如果超过这个范围则会发生溢出，无法到达正确的计算结果。

同时我们采用了非流水线设计，其吞吐量要远小于流水线设计，后续我们应当尝试改进来提升处理器的性能。

Conclusion

本次项目成功设计并实现了一款基于 FPGA 的 16 位 RISC-V CPU。我们采用了模块化的设计方法，构建了包括程序计数器（PC）单元、随机访问存储器（RAM）、指令解码器、控制单元、寄存器文件和算术逻辑单元（ALU）在内的多个关键组件。

系统架构的设计充分考虑了从指令获取、解码到执行以及结果写回等各个环节的需求。指令集架构设计方面，我们采用了固定 16 位指令长度，并提供了丰富的指令类型，以支持算术运算、逻辑运算、数据传输以及控制流程等基本操作。通过各个模块的协同工作，CPU 能够有效地执行各类指令。

在实现过程中，我们进行了仿真和上板测试，以验证 CPU 的功能和性能。仿真结果显示，CPU 能够准确地执行指令，例如计算累加和累乘等操作，并且结果与预期值完全一致。在此基础上，我们将设计实际部署到 FPGA 上，通过数码管显示

计算结果，进一步验证了设计的功能。

然而，项目也有许多未来可以改进的地方。当前设计采用的是非流水线架构，这在一定程度上限制了 CPU 的性能，尤其是在吞吐量方面。为了提升 CPU 的能力，未来的工作可以集中在实现流水线架构上，以提高指令的执行速度和效率。此外，16 位的寄存器宽度也对计算范围产生了一定的限制。探索扩展寄存器宽度的方法，或者实现动态溢出处理机制，将显著扩大 CPU 的应用范围。最后，对控制单元和 ALU 进行优化，以支持更复杂的运算，并进一步改进整体设计，以提高资源利用率和速度，也是需要研究的方向。通过这些改进，CPU 将能够更好地满足更多样化应用的需求。

Appendix

VHDL 模块实现与代码

CPU 顶层模块：

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity cpu is
    Port (
        clk_in : in STD_LOGIC;
        reset : in STD_LOGIC;
        seg : out std_logic_vector (7 downto 0); -- 数据端（最高位
为小数点）
        an : out std_logic_vector (7 downto 0) -- 阳极选择端
    );
end cpu;

architecture Behavioral of cpu is
    -- COMPONENT DECLARATION --

    component registerfile
    Port ( clk_in : in  STD_LOGIC;
        enable_in : in  STD_LOGIC;
        write_enable_in : in  STD_LOGIC;
        rA_data_out : out  STD_LOGIC_VECTOR (15 downto 0);
        rB_data_out : out  STD_LOGIC_VECTOR (15 downto 0);
```



```

        rD_data_in : in  STD_LOGIC_VECTOR (15 downto 0);
        sel_rA_in  : in  STD_LOGIC_VECTOR (2  downto 0);
        sel_rB_in  : in  STD_LOGIC_VECTOR (2  downto 0);
        sel_rD_in  : in  STD_LOGIC_VECTOR (2  downto 0);
        reg_0_out  : out STD_LOGIC_VECTOR (15 downto 0)
    );
end component;

component decoder
Port ( clk_in : in  STD_LOGIC;
      enable_in : in  STD_LOGIC;
      instruction_in : in  STD_LOGIC_VECTOR (15 downto 0);
      alu_op_out : out STD_LOGIC_VECTOR (4  downto 0);
      imm_data_out : out STD_LOGIC_VECTOR (7  downto 0);
      write_enable_out : out STD_LOGIC;
      sel_rA_out : out STD_LOGIC_VECTOR (2  downto 0);
      sel_rB_out : out STD_LOGIC_VECTOR (2  downto 0);
      sel_rD_out : out STD_LOGIC_VECTOR (2  downto 0)
    );
end component;

component alu
Port ( clk_in : in  STD_LOGIC;
      enable_in : in  STD_LOGIC;
      alu_op_in : in  STD_LOGIC_VECTOR (4  downto 0);
      pc_in : in  STD_LOGIC_VECTOR (15 downto 0);
      rA_data_in : in  STD_LOGIC_VECTOR (15 downto 0);
      rB_data_in : in  STD_LOGIC_VECTOR (15 downto 0);
      imm_data_in : in  STD_LOGIC_VECTOR (7  downto 0);
      result_out : out STD_LOGIC_VECTOR (15 downto 0);
      branch_out : out STD_LOGIC;
      rD_write_enable_in : in  STD_LOGIC;
      rD_write_enable_out : out STD_LOGIC
    );
end component;

```

```

component controlunit
Port ( clk_in : in  STD_LOGIC;
      reset_in : in  STD_LOGIC;
      alu_op_in : in  STD_LOGIC_VECTOR (4 downto 0);
      stage_out : out  STD_LOGIC_VECTOR (5 downto 0)
      );
end component;

component pcunit
Port ( clk_in : in  STD_LOGIC;
      pc_op_in : in  STD_LOGIC_VECTOR (1 downto 0);
      pc_in : in  STD_LOGIC_VECTOR (15 downto 0);
      pc_out : out  STD_LOGIC_VECTOR (15 downto 0)
      );
end component;

component ram
Port ( clk_in : in  STD_LOGIC;
      reset : in  STD_LOGIC;
      write_enable_in : in  STD_LOGIC;
      address_in : in  STD_LOGIC_VECTOR (15 downto 0);
      data_in : in  STD_LOGIC_VECTOR (15 downto 0);
      data_out : out  STD_LOGIC_VECTOR (15 downto 0)
      );
end component;

component SevenSegDisplay is
Port(clk : in  std_logic;           -- 100MHz 系统时钟
     en_n : in std_logic_vector (7 downto 0);-- 显示使能端，低电平
使能
     dp : in std_logic_vector (7 downto 0);      -- 小数点控制端
     number : in  std_logic_vector (31 downto 0); -- 8 个数字的输入数据（每个数字 4 位，共 8*4=32 位）

```

```
        seg : out std_logic_vector (7 downto 0);-- 数据端（最高位为小  
数点）
```

```
        an : out std_logic_vector (7 downto 0));    -- 阳极选择端  
end component;
```

```
-- SIGNAL DECLARATION --
```

```
-- REGFILE --
```

```
signal reg_enable : std_logic := '0';  
signal reg_write_enable : std_logic := '0';  
signal rA_data : std_logic_vector (15 downto 0) := (others => '0');  
signal rB_data : std_logic_vector (15 downto 0) := (others => '0');  
signal rD_data : std_logic_vector (15 downto 0) := (others => '0');  
signal rA_sel : std_logic_vector (2 downto 0) := (others => '0');  
signal rB_sel : std_logic_vector (2 downto 0) := (others => '0');  
signal rD_sel : std_logic_vector (2 downto 0) := (others => '0');  
signal reg_0_out : std_logic_vector (31 downto 0) := (others => '0');
```

```
-- ALU --
```

```
signal result : std_logic_vector (15 downto 0) := (others => '0');  
signal branch : std_logic := '0';  
signal rD_write_enable : std_logic := '0';  
signal alu_write_enable_out : std_logic := '0';
```

```
-- PCUNIT --
```

```
signal pc_op : std_logic_vector (1 downto 0) := (others => '0');  
signal pc_in : std_logic_vector (15 downto 0) := (others => '0');  
signal pc_out : std_logic_vector (15 downto 0) := (others => '0');
```

```
-- RAM --
```

```
signal ram_write_enable : std_logic := '0';
```

```

    signal address : std_logic_vector (15 downto 0) := (others => '0');
    signal ram_data_in : std_logic_vector (15 downto 0) := (others => '0');
    signal ram_data_out : std_logic_vector (15 downto 0) := (others =>
'0');

    -- PIPELINE --

    signal fetch_enable : std_logic := '0';
    signal reg_read : std_logic := '0';
    signal decoder_enable : std_logic := '0';
    signal alu_enable : std_logic := '0';
    signal ram_enable : std_logic := '0';
    signal reg_write : std_logic := '0';

    -- OTHERS --

    signal instruction : std_logic_vector (15 downto 0) := (others =>
'0');
    signal alu_op : std_logic_vector (4 downto 0) := (others => '0'); --
DECODER + ALU + CONTROLUNIT
    signal immediate : std_logic_vector (7 downto 0) := (others => '0');
-- DECODER + ALU
    signal cpu_reset : std_logic := '0'; -- CONTROLUNIT + RAM
    signal stage : std_logic_vector (5 downto 0) := (others => '0'); --
CONTROLUNIT
    signal reg_0 : STD_LOGIC_VECTOR (15 downto 0); -- cpu to
sevensegdisplay
    signal number_in : std_logic_vector (31 downto 0); -- cpu to
sevensegdisplay

begin

    cpu_registerfile : registerfile PORT MAP (
        clk_in => clk_in,

```



```

    enable_in => reg_enable,
    write_enable_in => reg_write_enable,
    rA_data_out => rA_data,
    rB_data_out => rB_data,
    rD_data_in => rD_data,
    sel_rA_in => rA_sel,
    sel_rB_in => rB_sel,
    sel_rD_in => rD_sel,
    reg_0_out => reg_0
);

```

```

cpu_decoder : decoder PORT MAP (
    clk_in => clk_in,
    enable_in => decoder_enable,
    instruction_in => instruction,
    alu_op_out => alu_op,
    imm_data_out => immediate,
    write_enable_out => rD_write_enable,
    sel_rA_out => rA_sel,
    sel_rB_out => rB_sel,
    sel_rD_out => rD_sel
);

```

```

cpu_alu : alu PORT MAP (
    clk_in => clk_in,
    enable_in => alu_enable,
    alu_op_in => alu_op,
    pc_in => pc_out,
    rA_data_in => rA_data,
    rB_data_in => rB_data,
    imm_data_in => immediate,
    result_out => result,
    branch_out => branch,
    rD_write_enable_in => rD_write_enable,
    rD_write_enable_out => alu_write_enable_out
);

```

```
);
```

```
cpu_controlunit : controlunit PORT MAP (  
    clk_in => clk_in,  
    reset_in => reset,  
    alu_op_in => alu_op,  
    stage_out => stage  
);
```

```
cpu_pcunit : pcunit PORT MAP (  
    clk_in => clk_in,  
    pc_op_in => pc_op,  
    pc_in => pc_in,  
    pc_out => pc_out  
);
```

```
cpu_ram : ram PORT MAP (  
    clk_in => clk_in,  
    reset => reset,  
    write_enable_in => ram_write_enable,  
    address_in => address,  
    data_in => ram_data_in,  
    data_out => ram_data_out  
);
```

```
cpu_sevensegdisplay : SevenSegDisplay PORT MAP (  
    clk => clk_in,  
    en_n => "00000000",  
    dp => "11111111",  
    number => number_in,  
    seg => seg,  
    an => an  
);
```

```
-- REGFILE --
```

```

reg_enable <= reg_read or reg_write;
reg_write_enable <= alu_write_enable_out and reg_write;

-- PIPELINE --

fetch_enable <= stage(0); -- FETCH
decoder_enable <= stage(1); -- DECODE
reg_read <= stage(2); -- REGISTER READ
alu_enable <= stage(3); -- EXECUTE
ram_enable <= stage(4); -- MEMORY
reg_write <= stage(5); -- REGISTER WRITE

-- PC --

pc_op <= "00" when cpu_reset = '1' else -- RESET
        "01" when branch = '0' and stage(5) = '1' else -- INCREMENT
        "10" when branch = '1' and stage(5) = '1' else -- BRANCH
        "11"; -- NOP

pc_in <= result;

-- MEMORY --

address <= result when ram_enable = '1' else pc_out; -- RAM ACCESS
MEMORY/FETCH STAGE

ram_data_in <= rB_data; -- rB = DATA, rA = ADDRESS, FOR ST OPERATION
ram_write_enable <= '1' when ram_enable = '1' and alu_op(3 downto 0)
= "1101" else '0'; -- RAM ENABLE + ST OPERATION

rD_data <= ram_data_out when reg_write = '1' and alu_op(3 downto 0)
= "1100" else result; -- REG DATA IS RAM WHEN LD OR ALU RESULT

instruction <= ram_data_out when fetch_enable = '1'; -- DATA FROM RAM
GOES TO DECODER

```

```
-- SEVENSEGDISPLAY --
```

```
    number_in <= (others=>'0') when cpu_reset = '1' else "000"&reg_0(7)
& "000"&reg_0(6) & "000"&reg_0(5) & "000"&reg_0(4) & "000"&reg_0(3) &
"000"&reg_0(2) & "000"&reg_0(1) & "000"&reg_0(0);
    end Behavioral;
```

Register File 模块 :

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;
```

```
entity registerfile is
```

```
    Port ( clk_in : in  STD_LOGIC;
          enable_in : in  STD_LOGIC;
          write_enable_in : in  STD_LOGIC;
          rA_data_out : out  STD_LOGIC_VECTOR (15 downto 0);
          rB_data_out : out  STD_LOGIC_VECTOR (15 downto 0);
          rD_data_in : in  STD_LOGIC_VECTOR (15 downto 0);
          sel_rA_in : in  STD_LOGIC_VECTOR (2 downto 0);
          sel_rB_in : in  STD_LOGIC_VECTOR (2 downto 0);
          sel_rD_in : in  STD_LOGIC_VECTOR (2 downto 0);
          reg_0_out : out STD_LOGIC_VECTOR (15 downto 0)
        );
```

```
end registerfile;
```

```
architecture Behavioral of registerfile is
```

```
    type reg_array is array(0 to 7) of STD_LOGIC_VECTOR (15 downto 0);
    signal reg_file: reg_array := (others => x"0000");
begin
    process(clk_in)
    begin
        if rising_edge(clk_in) then
            if enable_in = '1' then
```

```

        rA_data_out <=
reg_file(to_integer(unsigned(sel_rA_in)));
        rB_data_out <=
reg_file(to_integer(unsigned(sel_rB_in)));

        if write_enable_in = '1' then
            reg_file(to_integer(unsigned(sel_rD_in))) <=
rD_data_in;
        end if;
    end if;
end if;
end process;

reg_0_out <= reg_file(0);
end Behavioral;
```

此代码定义了寄存器组行为，包括时钟控制、使能信号响应、寄存器选择与写入操作，确保数据正确读写。

ALU 模块：

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity alu is
    Port ( clk_in : in  STD_LOGIC;
          enable_in : in  STD_LOGIC;
          alu_op_in : in  STD_LOGIC_VECTOR (4 downto 0);
          pc_in : in  STD_LOGIC_VECTOR (15 downto 0);
          rA_data_in : in  STD_LOGIC_VECTOR (15 downto 0);
          rB_data_in : in  STD_LOGIC_VECTOR (15 downto 0);
          imm_data_in : in  STD_LOGIC_VECTOR (7 downto 0);
          result_out : out  STD_LOGIC_VECTOR (15 downto 0);
          branch_out : out  STD_LOGIC;
          rD_write_enable_in : in  STD_LOGIC;
          rD_write_enable_out : out  STD_LOGIC);
```

```

end alu;

architecture Behavioral of alu is
    signal s_result: STD_LOGIC_VECTOR(17 downto 0) := (others => '0');
    signal s_shouldBranch: STD_LOGIC := '0';

    constant CMP_BIT_EQ: integer := 15;
    constant CMP_BIT_AGB: integer := 14;
    constant CMP_BIT_ALB: integer := 13;
    constant CMP_BIT_AZ: integer := 12;
    constant CMP_BIT_BZ: integer := 11;

begin
    process(clk_in, enable_in)
    begin
        if rising_edge(clk_in) and enable_in = '1' then
            rD_write_enable_out <= rD_write_enable_in;

            case alu_op_in(3 downto 0) is
                when "0000" => -- ADD
                    if alu_op_in(4) = '0' then
                        s_result(16 downto 0) <=
std_logic_vector(unsigned('0' & rA_data_in) + unsigned('0' &
rB_data_in));
                    else
                        s_result(16 downto 0) <=
std_logic_vector(signed(rA_data_in(15) & rA_data_in) +
signed(rB_data_in(15) & rB_data_in));
                    end if;
                    s_shouldBranch <= '0';
                when "0001" => -- SUB
                    if alu_op_in(4) = '0' then
                        s_result(16 downto 0) <=
std_logic_vector(unsigned('0' & rA_data_in) - unsigned('0' &
rB_data_in));

```



```

        else
            s_result(16 downto 0) <=
std_logic_vector(signed(rA_data_in(15) & rA_data_in) -
signed(rB_data_in(15) & rB_data_in));
        end if;
        s_shouldBranch <= '0';
when "0010" => -- NOT
    s_result(15 downto 0) <= not rA_data_in;
    s_shouldBranch <= '0';
when "0011" => -- AND
    s_result(15 downto 0) <= rA_data_in and rB_data_in;
    s_shouldBranch <= '0';
when "0100" => -- OR
    s_result(15 downto 0) <= rA_data_in or rB_data_in;
    s_shouldBranch <= '0';
when "0101" => -- XOR
    s_result(15 downto 0) <= rA_data_in xor rB_data_in;
    s_shouldBranch <= '0';
--
when "0110" => -- LSL
--
    result_out <=
std_logic_vector(shift_left(unsigned(rA_data_in),
to_integer(unsigned(rB_data_in(3 downto 0))))));
--
    s_shouldBranch <= '0';
--
when "0111" => -- LSR
--
    result_out <=
std_logic_vector(shift_right(unsigned(rA_data_in),
to_integer(unsigned(rB_data_in(3 downto 0))))));
--
    s_shouldBranch <= '0';
when "1000" => -- CMP
    if rA_data_in = rB_data_in then
        s_result(CMP_BIT_EQ) <= '1';
    else
        s_result(CMP_BIT_EQ) <= '0';
    end if;

```

```

        if rA_data_in = x"0000" then
            s_result(CMP_BIT_AZ) <= '1';
        else
            s_result(CMP_BIT_AZ) <= '0';
        end if;

        if rB_data_in = x"0000" then
            s_result(CMP_BIT_BZ) <= '1';
        else
            s_result(CMP_BIT_BZ) <= '0';
        end if;

        if alu_op_in(4) = '0' then
            if unsigned(rA_data_in) > unsigned(rB_data_in)
then
                s_result(CMP_BIT_AGB) <= '1';
            else
                s_result(CMP_BIT_AGB) <= '0';
            end if;

            if unsigned(rA_data_in) < unsigned(rB_data_in)
then
                s_result(CMP_BIT_ALB) <= '1';
            else
                s_result(CMP_BIT_ALB) <= '0';
            end if;

        else
            if signed(rA_data_in) > signed(rB_data_in) then
                s_result(CMP_BIT_AGB) <= '1';
            else
                s_result(CMP_BIT_AGB) <= '0';
            end if;

            if signed(rA_data_in) < signed(rB_data_in) then

```

```

        s_result(CMP_BIT_ALB) <= '1';
    else
        s_result(CMP_BIT_ALB) <= '0';
    end if;
end if;

s_result(10 downto 0) <= (others => '0');
s_shouldBranch <= '0';

when "1001" => -- B
    if alu_op_in(4) = '1' then
        s_result(15 downto 0) <= rA_data_in;
    else
        s_result(15 downto 0) <= x"00" & imm_data_in;
    end if;
    s_shouldBranch <= '1';
when "1010" => -- BEQ
    if s_result(CMP_BIT_EQ) = '1' then
        s_result(15 downto 0) <= rA_data_in;
        s_shouldBranch <= '1';
    else
        s_shouldBranch <= '0';
    end if;
when "1011" => -- IMMEDIATE
    if alu_op_in(4) = '1' then
        s_result(15 downto 0) <= imm_data_in & x"00";
    else
        s_result(15 downto 0) <= x"00" & imm_data_in;
    end if;
    s_shouldBranch <= '0';
when "1100" => -- LOAD
    s_result(15 downto 0) <= rA_data_in;
    s_shouldBranch <= '0';
when "1101" => -- STORE
    s_result(15 downto 0) <= rA_data_in;

```

```

        s_shouldBranch <= '0';
    when "1110" => -- multiply
        if alu_op_in(4) = '0' then
            s_result(16 downto 0) <=
std_logic_vector(TO_UNSIGNED(TO_INTEGER(unsigned('0' & rA_data_in))
*
TO_INTEGER(unsigned('0' & rB_data_in)), 17));
        else
            s_result(16 downto 0) <=
std_logic_vector(TO_SIGNED(TO_INTEGER(signed('0' & rA_data_in))
*
TO_INTEGER(signed('0' & rB_data_in)), 17));
        end if;
        s_shouldBranch <= '0';
    when others =>
        s_result(15 downto 0) <= x"DEAD"; -- ERROR
    end case;
end if;
end process;

result_out <= s_result(15 downto 0);
branch_out <= s_shouldBranch;

```

end Behavioral;

ALU 根据操作码执行不同运算,涵盖加减法和逻辑运算,并输出结果与零标志位,为程序分支等操作提供判断依据。

PC Unit 模块:

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity pcunit is
    Port ( clk_in : in  STD_LOGIC;
          pc_op_in : in  STD_LOGIC_VECTOR (1 downto 0);
          pc_in : in  STD_LOGIC_VECTOR (15 downto 0);
          pc_out : out  STD_LOGIC_VECTOR (15 downto 0));

```

```

end pcunit;

architecture Behavioral of pcunit is
    signal pc: std_logic_vector (15 downto 0) := x"0000";

begin
    process (clk_in)
    begin
        if rising_edge(clk_in) then
            case pc_op_in is
                when "00" => -- RESET
                    pc <= x"0000";
                when "01" => -- INCREMENT
                    pc <= std_logic_vector(unsigned(pc) + 1);
                when "10" => -- BRANCH
                    pc <= pc_in;
                when "11" => -- HOLD
                    null;
                when others =>
                    pc <= pc;
            end case;
        end if;
    end process;

    pc_out <= pc;

end Behavioral;

```

此代码根据控制信号 pc_opcode 对程序计数器进行管理。当 pc_opcode 设置为不同值时，分别实现 PC 增 1、更新为新值、保持不变或复位至初始地址，确保程序流程的正确控制。

RAM 模块:

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity ram is
    Port ( clk_in : in  STD_LOGIC;
          reset : in  STD_LOGIC;
          write_enable_in : in  STD_LOGIC;
          address_in : in  STD_LOGIC_VECTOR (15 downto 0);

```

```

        data_in : in  STD_LOGIC_VECTOR (15 downto 0);
        data_out : out STD_LOGIC_VECTOR (15 downto 0)
    );
end ram;

-- 累加汇编代码
--IMMEDIATE rA, 0
--IMMEDIATE rB, 1
--IMMEDIATE rC, N
--Loop_Start_Label:
--    ; 循环体代码
--    CMP rB, rC
--    BEQ rD, 1, End_Label
--    ADD rA, rA, rB
--    ADD rB, rB, 1
--    B Loop_Start_Label

--End_Label:
--    ; 循环结束后代码

architecture Behavioral of ram is
    type ram_array is array (0 to 255) of STD_LOGIC_VECTOR (15 downto 0);
    signal ram : ram_array := ( -- RAM preload instructions
--
--            -----累加-----
--            '0' & "1011" & "000" & x"01",          -- IMMEDIATE
rA, 1
--            '0' & "1011" & "001" & x"02",          -- IMMEDIATE
rB, 2
--            '0' & "1011" & "010" & x"06",          -- IMMEDIATE
rC, N
--            '0' & "1011" & "100" & x"01",          -- IMMEDIATE
r4, 1
--            '0' & "1011" & "101" & x"0B",          -- IMMEDIATE
r5, End_Label
--            '0' & "1011" & "110" & x"06",          -- IMMEDIATE

```

```

r6, Loop_start_Label
--          '0' & "0000" & "000" & "000" & "001" & "00", -- ADD rA, rA,
rB
--          '0' & "1000" & "011" & "001" & "010" & "00", -- CMP rB, rC
--          '0' & "1010" & "000" & "101" & "000" & "00", -- BEQ rD, 1,
End_Label
--          '0' & "0000" & "001" & "001" & "100" & "00", -- ADD rB, rB,
r4(1)
--          '1' & "1001" & "000" & "110" & "000" & "00", -- B
Loop_Start_Label
--          -----
--          -----累乘-----
--          '0' & "1011" & "000" & x"01", -- IMMEDIATE
rA, 1
--          '0' & "1011" & "001" & x"02", -- IMMEDIATE
rB, 2
--          '0' & "1011" & "010" & x"05", -- IMMEDIATE
rC, N
--          '0' & "1011" & "100" & x"01", -- IMMEDIATE
r4, 1
--          '0' & "1011" & "101" & x"0B", -- IMMEDIATE
r5, End_Label
--          '0' & "1011" & "110" & x"06", -- IMMEDIATE
r6, Loop_start_Label
--          '0' & "1110" & "000" & "000" & "001" & "00", -- MUL rA, rA,
rB
--          '0' & "1000" & "011" & "001" & "010" & "00", -- CMP rB, rC
--          '0' & "1010" & "000" & "101" & "000" & "00", -- BEQ rD, 1,
End_Label
--          '0' & "0000" & "001" & "001" & "100" & "00", -- ADD rB, rB,
r4(1)
--          '1' & "1001" & "000" & "110" & "000" & "00", -- B
Loop_Start_Label
--          -----
--          '0' & "1110" & "010" & "001" & "000" & "00", -- MUL r2, r1,

```

```

r0
--      '0' & "1011" & "000" & x"0A", -- imm r0 = 0x000A (580A) ,
0
--      '0' & "1011" & "001" & x"AA", -- imm r1 = 0x00AA (59AA) ,
1
--      '0' & "0000" & "010" & "001" & "000" & "00",
--      '0' & "0010" & "001" & "001" & "00000", -- not r1 = xFF55
(1120) , 2
--      '0' & "0000" & "010" & "001" & "000" & "00", -- nor r1 = xFF55
(1120) , 3
--      '0' & "1011" & "011" & x"08", -- imm r3 = 0x008 (5B08) ,
4
--      '0' & "0110" & "010" & "010" & "011" & "00", -- LSL r2 by
r3, r2 = x5F00 (324C) , 5
--      '0' & "1011" & "100" & x"04", -- imm r4 = 0x04 (5C04) , 6
--      '0' & "0111" & "010" & "010" & "100" & "00", -- LSR r2 by
r4, r2 = x05F0 , 7
--      '1' & "1001" & "000" & x"0B", -- Branch to ram address 11
(C80B) , 8
--      x"0000", -- NOP (0000) , 9
--      x"0000", -- NOP (0000) , 10
--      '0' & "0001" & "101" & "010" & "011" & "00", -- unsigned
SUB r5 = r2 - r3, r5 = x05E8 (0D4C) , 11
--      '0' & "1011" & "110" & x"FF", -- imm r6 = 0xFF (5EFF), 12
--      '0' & "1101" & "000" & "110" & "101" & "00", -- ST r5 (rA)
data at r6 (rB) address (x05E8 at xFF) (68D4) , 13
--      '0' & "1100" & "111" & "110" & "00000", -- LD r6 address
into r7, r7 = x05E8 (67C0) , 14
--      '0' & "1000" & "000" & "101" & "111" & "00", -- unsigned
CMP r0 = r5 ? r7, r0 = x"4000" (40BC) , 15
--      '0' & "1011" & "001" & x"18", -- imm r1 = 0x18 (5918) , 16
--      '0' & "1010" & "000" & "001" & "000" & "00", -- BEQ r1 =
r0, r1 = address, r0 = out from cmp (5020) , 17
--      x"0000", -- NOP (0000) , 18
--      x"0000", -- NOP (0000) , 19

```



```

--          x"0000", -- NOP (0000) , 20
--          x"0000", -- NOP (0000) , 21
--          x"0000", -- NOP (0000) , 22
--          x"0000", -- NOP (0000) , 23
--          '0' & "0011" & "010" & "011" & "001" & "00", -- r2 = r3 AND
r1, r2 = x09 (1A64) , 24
--          '0' & "0100" & "011" & "010" & "001" & "00", -- r3 = r2 OR
r1, r3 = x14 (2344) , 25
--          '0' & "0101" & "100" & "011" & "011" & "00", -- r4 = r3 XOR
r3, r4 = 0x00 (2C6C) , 26

--          others => x"0000"
--          others => '0' & "0000" & "111" & "111" & "111" & "00"
);
begin
    process(clk_in)
    begin
        if rising_edge(clk_in) then
            if reset = '1' then
                -- ram <= (others => x"0000");
                null;
            else
                if write_enable_in = '1' then
                    ram(to_integer(unsigned(address_in(7 downto 0)))) <=
data_in;
                end if;
            end if;
        end if;
    end process;

    data_out <= ram(to_integer(unsigned(address_in(7 downto 0))));

end Behavioral;

```

此代码实现了一个简单的 RAM 模块,通过 enable 和 wr_enable 信号控制数据的读写操作, address 信号指定访问的内存地址, data_in 为写入数据,

data_out 输出读取数据，满足程序和数据存储需求。

Decoder 模块:

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity decoder is
    Port ( clk_in : in  STD_LOGIC;
          enable_in : in  STD_LOGIC;
          instruction_in : in  STD_LOGIC_VECTOR (15 downto 0);
          alu_op_out : out  STD_LOGIC_VECTOR (4 downto 0);
          imm_data_out : out  STD_LOGIC_VECTOR (7 downto 0);
          write_enable_out : out  STD_LOGIC;
          sel_rA_out : out  STD_LOGIC_VECTOR (2 downto 0);
          sel_rB_out : out  STD_LOGIC_VECTOR (2 downto 0);
          sel_rD_out : out  STD_LOGIC_VECTOR (2 downto 0));
end decoder;
```

architecture Behavioral of decoder is

```
begin
    process(clk_in)
    begin
        if rising_edge(clk_in) and enable_in = '1' then
            alu_op_out <= instruction_in(15 downto 11);
            imm_data_out <= instruction_in(7 downto 0);
            sel_rA_out <= instruction_in(7 downto 5);
            sel_rB_out <= instruction_in(4 downto 2);
            sel_rD_out <= instruction_in(10 downto 8);

            case instruction_in(14 downto 11) is
                when "1001" =>
                    write_enable_out <= '0';
                when "1010" =>
```

```

        write_enable_out <= '0';
    when "1101" =>
        write_enable_out <= '0';
    when others =>
        write_enable_out <= '1';
    end case;
end if;
end process;

```

end Behavioral;

此代码根据指令的操作码对指令进行解码，提取出操作码 alu_op、写入使能信号 wr_enable、寄存器选择信号 rA、rB、rD 以及立即数 imm，为后续指令执行提供必要控制信息和操作数选择。

Control Unit 模块:

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity controlunit is
    Port ( clk_in : in  STD_LOGIC;
          reset_in : in  STD_LOGIC;
          alu_op_in : in  STD_LOGIC_VECTOR (4 downto 0);
          stage_out : out STD_LOGIC_VECTOR (5 downto 0));
end controlunit;

architecture Behavioral of controlunit is
    signal stage: std_logic_vector(5 downto 0) := "000001";

begin
    process(clk_in)
    begin
        if rising_edge(clk_in) then
            if reset_in = '1' then
                stage <= "000001";
            else
                case stage is
                    when "000001" => -- FETCH
                        stage <= "000010";
                    when "000010" => -- DECODE
                        stage <= "000100";

```

```

        when "000100" => -- REG READ
            stage <= "001000";
        when "001000" => -- ALU
            if alu_op_in(3 downto 0) = "1100" or alu_op_in(3
downto 0) = "1101" then -- LOAD OR STORE
                stage <= "010000";
            else
                stage <= "100000";
            end if;
        when "010000" => -- MEMORY
            stage <= "100000";
        when "100000" => -- REG WRITE
            stage <= "000001";
        when others =>
            stage <= "000001";
        end case;
    end if;
end if;
end process;

stage_out <= stage;

```

end Behavioral;

此代码实现了一个简单的状态机作为控制单元，通过时钟信号和复位信号控制状态的切换，每个状态对应指令执行流程中的一个阶段（如取指、解码、执行、写回等），为各模块的协调工作提供时序控制信号。

Sevensegdisplay 模块:

```

library ieee;
use ieee.std_logic_1164.all;
use IEEE.std_logic_unsigned.all;
use IEEE.std_logic_arith.all;

entity SevenSegDisplay is
    Port(clk : in  std_logic;                -- 100MHz 系统时
钟
        en_n : in std_logic_vector (7 downto 0);    -- 显示使能端，
低电平使能
        dp : in std_logic_vector (7 downto 0);      -- 小数点控制端
        number : in  std_logic_vector (31 downto 0); -- 8 个数字的输

```

```

入数据（每个数字 4 位，共 8*4=32 位）
    seg : out  std_logic_vector (7 downto 0);    -- 数据端（最高
位为小数点）
    an : out  std_logic_vector (7 downto 0));    -- 阳极选择端
end SevenSegDisplay;

architecture Behavioral of SevenSegDisplay is

    signal cnt: std_logic_vector (19 downto 0);
    signal anode_select: std_logic_vector (2 downto 0); -- 3 位阳极扫描信号，也是状态机状态
    signal digit_data: std_logic_vector (3 downto 0); -- 当前显示的数字数据
    signal dp_bit: std_logic;                        -- 当前小数点状态
    signal segment_data: std_logic_vector (6 downto 0); -- 七段数码管数据信号（不含小数点）
    signal an_tmp: std_logic_vector (7 downto 0);    -- 未与使能端组合前的阳极扫描信号（8 位）

begin
    -- 计数器
    process(clk)
    begin
        if rising_edge(clk) then
            cnt <= cnt + '1';
        end if;
    end process;

    -- 最高三位，作为扫描状态（每个数字刷新率：100MHz/220 = 95.3Hz）
    anode_select <= cnt(19 downto 17);

    -- 阳极扫描（3-8 译码器）
    with anode_select select
        an_tmp <=

```

```

        "11111110" when "000",
        "11111101" when "001",
        "11111011" when "010",
        "11110111" when "011",
        "11101111" when "100",
        "11011111" when "101",
        "10111111" when "110",
        "01111111" when "111",
        "11111111" when others;

```

-- 应用低电平使能控制

```
an <= an_tmp or en_n;
```

-- 数据选择多路复用

```

with anode_select select
    digit_data <=
        number(3 downto 0)   when "000",
        number(7 downto 4)   when "001",
        number(11 downto 8)  when "010",
        number(15 downto 12) when "011",
        number(19 downto 16) when "100",
        number(23 downto 20) when "101",
        number(27 downto 24) when "110",
        number(31 downto 28) when "111",
        "0000"               when others;

```

-- 小数点位

```

with anode_select select
    dp_bit <=
        dp(0) when "000",
        dp(1) when "001",
        dp(2) when "010",
        dp(3) when "011",
        dp(4) when "100",
        dp(5) when "101",

```

```

        dp(6) when "110",
        dp(7) when "111",
        '1'   when others;

-- 十六进制到七段数码管译码 (gfedcba)
with digit_data select
    segment_data <=
        "1000000" when "0000", -- 0
        "1111001" when "0001", -- 1
        "0100100" when "0010", -- 2
        "0110000" when "0011", -- 3
        "0011001" when "0100", -- 4
        "0010010" when "0101", -- 5
        "0000010" when "0110", -- 6
        "1111000" when "0111", -- 7
        "0000000" when "1000", -- 8
        "0010000" when "1001", -- 9
        "0001000" when "1010", -- A
        "0000011" when "1011", -- b
        "1000110" when "1100", -- C
        "0100001" when "1101", -- d
        -- "1101010" when "1110", -- M
        -- "0001100" when "1111", -- P
        "0000110" when "1110", -- E
        "0001110" when "1111", -- F
        "1111111" when others; -- 全灭

-- 组合小数点和七段数据（小数点在最高位）
seg <= dp_bit & segment_data;

end Behavioral;
```